

# SPRING FRAMEWORK

---

## 1.What is Spring?

Spring Framework is a lightweight, open-source Java framework used to build scalable, maintainable, and enterprise-level applications.

It provides infrastructure support for:

- Web applications
- REST APIs
- Database integration
- Security
- Microservices

## Why is it called “Lightweight”?

- Does not require heavy Java EE containers
  - Uses POJOs (Plain Old Java Objects)
  - Minimal memory footprint
  - Modular architecture
-

## **2.Why Spring Was Introduced?**

Before Spring, enterprise applications used:

Enterprise JavaBeans (EJB)

Problems with EJB:

- Complex configuration
- Heavy container
- Hard to test
- Tightly coupled code

Spring solved this by:

- Promoting POJOs
- Using Dependency Injection
- Reducing configuration complexity
- Improving testability

---

## **3.Core Philosophy of Spring**

Spring is built around two major principles:

1. Inversion of Control (IoC)
  2. Dependency Injection (DI)
-

## 4.Design Principles (Foundation Before Spring)

Spring heavily follows **SOLID** principles.

### Single Responsibility Principle (SRP)

A class should have only one reason to change.

Real-time example:

- Report → generates report
  - ReportPrinter → prints report
- 

### Open Closed Principle (OCP)

Open for extension, closed for modification.

Example:

Add new PaymentMethod without modifying PaymentService.

---

### Dependency Inversion Principle (DIP)

Depend on abstractions, not implementations.

```
interface Database {  
    void connect();  
}
```

Instead of:

```
MySQLDatabase db = new MySQLDatabase();
```

Use:

```
Database db = new MySQLDatabase();
```

This is the foundation of Spring DI.

---

## **5.Inversion of Control (IoC)**

### **What is IoC?**

Normally, we create objects:

```
Engine engine = new PetrolEngine();
```

In IoC:

Spring creates the object.

Control of object creation is inverted from developer to framework.

---

## **6.Dependency Injection (DI)**

### **What is DI?**

Dependency Injection means:

A class receives its dependencies from outside instead of creating them internally.

### **Types of DI:**

1. Constructor Injection (Recommended)
  2. Setter Injection
  3. Field Injection
-

## Real-Time Example – Car & Engine

```
interface Engine {  
    void start();  
}  
  
class ElectricEngine implements Engine {  
    public void start() {  
        System.out.println("Electric Engine Started");  
    }  
}  
  
class Car {  
    private Engine engine;  
  
    public Car(Engine engine) {  
        this.engine = engine;  
    }  
  
    public void drive() {  
        engine.start();  
        System.out.println("Car is moving");  
    }  
}
```

Car depends on Engine abstraction → Loose coupling achieved.

---

## 7.What is IoC Container?

The IoC container is the core of Spring.

It is responsible for:

- Creating beans
- Injecting dependencies

- Managing lifecycle
  - Managing scope
- 

## **Two Important Containers**

### **1.BeanFactory**

Basic container

Provides lazy initialization

### **2.ApplicationContext (Most Used)**

Advanced container

Provides:

- Eager initialization
- Internationalization
- Event handling

Common implementations:

- ClassPathXmlApplicationContext
  - AnnotationConfigApplicationContext
  - FileSystemXmlApplicationContext
- 

## **8.What is a Bean?**

A Bean is simply:

An object managed by Spring IoC container.

---

## 9. Ways to Configure Beans

1. XML Configuration (Old)
  2. Annotation Configuration (Modern)
  3. Java-based Configuration
- 

## 10. Creating IoC Container Using XML

### Step 1 – Add Dependency

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-context</artifactId>
  <version>5.3.34</version>
</dependency>
```

---

### Step 2 – Create Bean

```
public class MessageService {
    public void sendMessage() {
        System.out.println("Message sent successfully!");
    }
}
```

---

### Step 3 – beans.xml

```
<bean id="msgService" class="MessageService"/>
```

---

### Step 4 – Load Container

```
ApplicationContext context =
    new ClassPathXmlApplicationContext("beans.xml");
```

```
MessageService service =
```

```
context.getBean("msgService", MessageService.class);  
  
service.sendMessage();
```

---

## Internal Flow of Spring

1. Load configuration
  2. Create bean
  3. Inject dependencies
  4. Store bean in container
  5. Return bean when requested
- 

## 11.Annotation-Based Configuration (Modern Approach)

---

### @Configuration

Marks a class as configuration class.

```
@Configuration
```

---

### @ComponentScan

Tells Spring where to scan for components.

```
@ComponentScan("com.example")
```

---

### @Component

Marks a class as Spring-managed bean.

```
@Component  
public class EmailService { }
```

---

## **@Autowired**

Injects dependency automatically.

```
@Autowired
private EmailService emailService;
```

---

## **@Bean**

Used inside @Configuration class.

```
@Bean
public Engine engine() {
    return new PetrolEngine();
}
```

Used when:

- Third-party class
  - Need manual control
- 

## **12.Stereotype Annotations**

| <b>Annotatio<br/>n</b> | <b>Layer</b>      |
|------------------------|-------------------|
| @Compon<br>ent         | Generic           |
| @Service               | Business<br>Layer |
| @Reposit<br>ory        | Data Layer        |
| @Controll<br>er        | Web Layer         |

---

## 13.Handling Multiple Beans

If two beans of same type exist:

Spring throws:

NoUniqueBeanDefinitionException

---

### **@Primary**

Marks default bean.

@Primary

@Component

class EmailService { }

---

### **@Qualifier**

Specifies exact bean to inject.

@Autowired

@Qualifier("smsService")

private NotificationService service;

---

## 14.Bean Scope

Default scope = Singleton

---

## Types of Scope

| Scope     | Meaning                  |
|-----------|--------------------------|
| singleton | One object per container |
| prototype | New object every time    |
| request   | One per HTTP request     |
| session   | One per HTTP session     |

---

## Example

```
@Scope("prototype")
@Component
public class ReceiptGenerator { }
```

---

## 15. @Value Annotation

Used to inject properties.

```
@Value("${app.name}")
private String appName;
```

Used with application.properties.

---

## 16.Bean Lifecycle

Spring manages full lifecycle of singleton beans.

---

## **@PostConstruct**

Runs after dependency injection.

```
@PostConstruct  
public void init() { }
```

---

## **@PreDestroy**

Runs before bean destruction.

```
@PreDestroy  
public void cleanup() { }
```

Works for singleton beans only.

---

## **17.Spring Core Packages**

Important packages:

- org.springframework.beans
- org.springframework.context

These provide IoC and DI functionality.

---

## **18. Real-Time Example – Payment System**

Imagine:

PaymentService depends on:

- PaymentGateway
- NotificationService

Spring:

- Creates all objects
- Injects dependencies
- Resolves ambiguity
- Manages lifecycle

No manual `new` keyword required.

---

## 19. Why Spring is Preferred in Industry

- Loose coupling
  - Easy testing (mocking dependencies)
  - Clean architecture
  - Modular structure
  - Production-ready ecosystem
- 

## 20. Spring Ecosystem Overview

Spring Core → IoC & DI

Spring Data → Database

Spring MVC → Web

Spring Security → Authentication

Spring Boot → Auto configuration

Spring Cloud → Microservices

Spring Core is the foundation of all modules.